

SKILL-Week 3:

TASK-1:

```
harshitha18@LAPTOP-6E22AR  X  +  v
harshitha18@LAPTOP-6E22AR6C:~$ cd ~/OSSP
nano task1.c
```

```
harshitha18@LAPTOP-6E22AR  X  +  v
GNU nano 8.7.1 task1.c
#include <stdio.h>

int main()
{
    int choice;

    while (1)
    {
        printf("\n==== MAIN MENU =====\n");
        printf("1. Say Hello\n");
        printf("2. Display Number\n");
        printf("0. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice == 1)
        {
            printf("Hello! Welcome.\n");
        }
        else if (choice == 2)
        {
            printf("Number = 100\n");
        }
        else if (choice == 0)
        {
            printf("Exiting Program...\n");
            break;
        }
        else
        {
            printf("Invalid Choice!\n");
        }
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute   ^C Location  ^U Undo      ^-A Set Mark  ^-] To Bracket ^-B Previous
^X Exit      ^R Read File  ^E Replace    ^P Paste      ^_ Justify   ^/_ Go To Line ^-R Redo     ^-G Copy      ^-_ Where Was  ^-F Next
```

```
harshitha18@LAPTOP-6E22AR6C:~$ cd ~/OSSP
nano task1.c
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ gcc task1.c -o task1
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ ./task1

===== MAIN MENU =====
1. Say Hello
2. Display Number
0. Exit
Enter your choice: 1
Hello! Welcome.

===== MAIN MENU =====
1. Say Hello
2. Display Number
0. Exit
Enter your choice: 2
Number = 100

===== MAIN MENU =====
1. Say Hello
2. Display Number
0. Exit
Enter your choice: 5
Invalid Choice!

===== MAIN MENU =====
1. Say Hello
2. Display Number
0. Exit
Enter your choice: 0
Exiting Program...
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ |
```

successfully implemented an interactive menu-driven program using a main loop to accept user input and perform actions. It also demonstrated how to handle user choices, validate input, and exit the program efficiently using control flow statements.

Task-2;

```
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ nano File1.txt
```

```
harshitha18@LAPTOP-6E22AR  x  +  v  GNU nano 8.7.1  File1.txt *
Hello, this is File1.
This content will be copied to File2.

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M-A Set Mark  ^M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_/ Go To Line ^E Redo       ^M-G Copy     ^M-B Where Was
```

```
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ nano copyfile.c
```

```
harshitha18@LAPTOP-6E22AR  x  +  v  GNU nano 8.7.1  copyfile.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int source, destination;
    char buffer[100];
    int bytesRead;

    source = open("File1.txt", O_RDONLY);

    if (source < 0) {
        printf("Error opening File1.txt\n");
        return 1;
    }

    destination = open("File2.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        printf("Error creating File2.txt\n");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {
        write(destination, buffer, bytesRead);
    }

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M-A Set Mark  ^M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_/ Go To Line ^E Redo       ^M-G Copy     ^M-B Where Was
```

```

harshitha18@LAPTOP-6E22AR6C:~/OSSP$ gcc copyfile.c -o copyfile
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ ./copyfile
File copied successfully.
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ cat File2.txt
Hello, this is File1.
This content will be copied to File2.
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ |

```

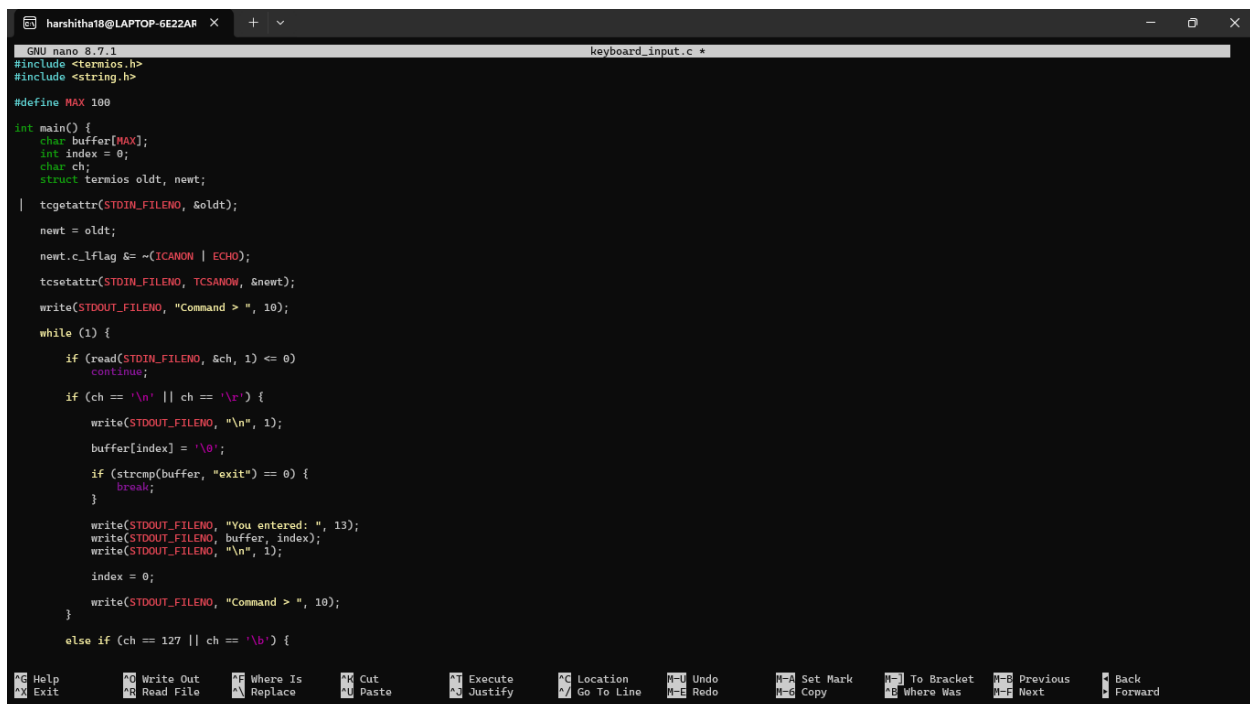
The contents of `File1.txt` were successfully copied to `File2.txt` using Linux file system calls. This task demonstrates the use of `open()`, `read()`, `write()`, and `close()` for basic file handling in C.

Task-3;

```

harshitha18@LAPTOP-6E22AR6C:~/OSSP$ nano keyboard_input.c

```



```

GNU nano 2.7.1 keyboard_input.c
#include <termios.h>
#include <string.h>

#define MAX 100

int main() {
    char buffer[MAX];
    int index = 0;
    char ch;
    struct termios oldt, newt;

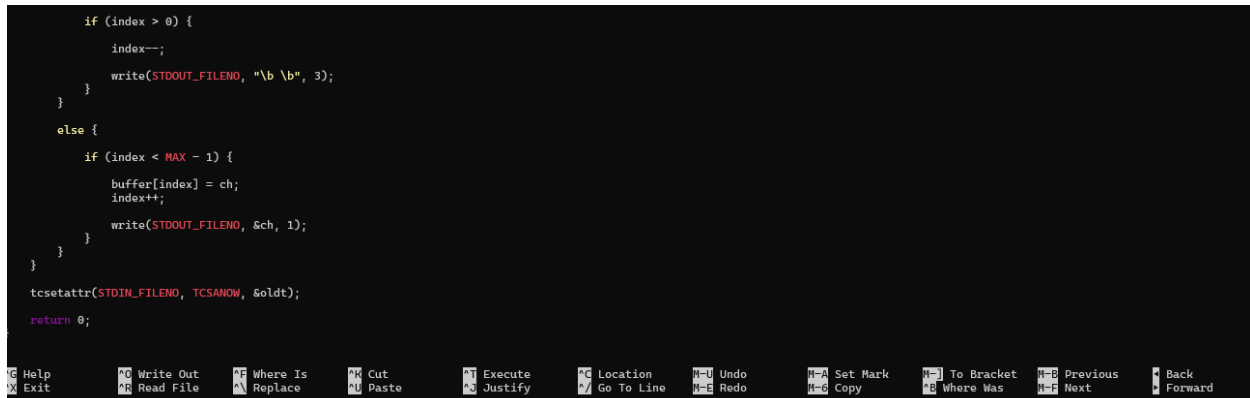
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    write(STDOUT_FILENO, "Command > ", 10);

    while (1) {
        if (read(STDIN_FILENO, &ch, 1) <= 0)
            continue;

        if (ch == '\n' || ch == '\r') {
            write(STDOUT_FILENO, "\n", 1);
            buffer[index] = '\0';
            if (strcmp(buffer, "exit") == 0) {
                break;
            }

            write(STDOUT_FILENO, "You entered: ", 13);
            write(STDOUT_FILENO, buffer, index);
            write(STDOUT_FILENO, "\n", 1);
            index = 0;
            write(STDOUT_FILENO, "Command > ", 10);
        }
        else if (ch == 127 || ch == '\b') {

```



```

            if (index > 0) {
                index--;
                write(STDOUT_FILENO, "\b\b", 3);
            }
        }
        else {
            if (index < MAX - 1) {
                buffer[index] = ch;
                index++;
                write(STDOUT_FILENO, &ch, 1);
            }
        }
    }

    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
    return 0;

```

```

harshitha18@LAPTOP-6E22AR6C:~/OSSP$ gcc keyboard_input.c -o keyboard_input
harshitha18@LAPTOP-6E22AR6C:~/OSSP$ ./keyboard_input
Command >
You entered:
Command > hello
You entered: hello
Command > exit

```

The program successfully captures keyboard input character-by-character and handles Enter, Backspace, and multiple-character commands using an input buffer.

It also uses terminal settings and `read()`/`write()` operations to provide controlled user interaction similar to a basic shell.

Task-4:

```

harshitha18@LAPTOP-6E22AR6C:~$ nano task4.c

```

```

GNU nano 2.9.3
task4.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <termios.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX 100
#define HISTORY_SIZE 10

int main() {
    char buffer[MAX];
    char history[HISTORY_SIZE][MAX];

    int index = 0;
    int history_count = 0;
    int history_pos = 0;

    char ch;
    struct termios oldt, newt;
    tcgetattr(STDIN_FILENO, &oldt);

    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1) {
        index = 0;
        buffer[0] = '\0';
        history_pos = history_count;

        write(STDOUT_FILENO, "Command > ", 10);

        while (1) {
            read(STDIN_FILENO, &ch, 1);

            /* Enter Key */
            if (ch == '\n' || ch == '\r') {
                buffer[index] = '\0';
                write(STDOUT_FILENO, "\n", 1);
                break;
            }

            /* Backspace */
            else if (ch == 127 || ch == '\b') {
                if (index > 0) {
                    index--;
                    buffer[index] = '\0';
                }
            }
        }
    }
}

```

```
harshitha18@LAPTOP-6E22AR X + - task4.c *
GNU nano 8.7.1

        write(STDOUT_FILENO, "\b \b", 3);
    }
}
/* Escape Sequence */
else if (ch == 27) {
    char seq[2];
    read(STDIN_FILENO, &seq[0], 1);
    read(STDIN_FILENO, &seq[1], 1);
    if (seq[0] == '[') {
        /* Up Arrow */
        if (seq[1] == 'A') {
            if (history_pos > 0) {
                history_pos--;
                while (index > 0) {
                    write(STDOUT_FILENO, "\b \b", 3);
                    index--;
                }
                strcpy(buffer, history[history_pos]);
                index = strlen(buffer);
                write(STDOUT_FILENO, buffer, index);
            }
        }
        /* Down Arrow */
        else if (seq[1] == 'B') {
            if (history_pos < history_count - 1) {
                history_pos++;
                while (index > 0) {
                    write(STDOUT_FILENO, "\b \b", 3);
                    index--;
                }
                strcpy(buffer, history[history_pos]);
                index = strlen(buffer);
                write(STDOUT_FILENO, buffer, index);
            }
        }
        else if (history_pos == history_count - 1) {
            history_pos = history_count;
        }
    }
}

|
⌕ Help      ⌕ Write Out  ⌕ Where Is  ⌕ Cut       ⌕ Execute  ⌕ Location  ⌕ Undo     ⌕ Set Mark  ⌕ To Bracket ⌕ Previous ⌕ Back      ⌕ Prev Word ⌕ Home
⌕ Exit      ⌕ Read File  ⌕ Replace  ⌕ Paste     ⌕ Justify  ⌕ Go To Line ⌕ Redo     ⌕ Copy      ⌕ Where Was ⌕ Next     ⌕ Forward  ⌕ Next Word ⌕ End
```

```
harshitha18@LAPTOP-6E22AR X + - task4.c *
GNU nano 8.7.1

        while (index > 0) {
            write(STDOUT_FILENO, "\b \b", 3);
            index--;
        }
        buffer[0] = '\0';
    }
}
/* Normal Character */
else {
    if (index < MAX - 1) {
        buffer[index++] = ch;
        buffer[index] = '\0';
        write(STDOUT_FILENO, &ch, 1);
    }
}
/* Exit command */
if (strcmp(buffer, "exit") == 0) {
    break;
}
if (strlen(buffer) == 0)
    continue;
/* Store command in history */
if (history_count < HISTORY_SIZE) {
    strcpy(history[history_count], buffer);
    history_count++;
}
/* Tokenize Command */
char commandCopy[MAX];
strcpy(commandCopy, buffer);
char *args[20];
int i = 0;
char *token = strtok(commandCopy, " ");
while (token != NULL && i < 19) {
    args[i++] = token;
    token = strtok(NULL, " ");
}

|
⌕ Help      ⌕ Write Out  ⌕ Where Is  ⌕ Cut       ⌕ Execute  ⌕ Location  ⌕ Undo     ⌕ Set Mark  ⌕ To Bracket ⌕ Previous ⌕ Back      ⌕ Prev Word ⌕ Home
⌕ Exit      ⌕ Read File  ⌕ Replace  ⌕ Paste     ⌕ Justify  ⌕ Go To Line ⌕ Redo     ⌕ Copy      ⌕ Where Was ⌕ Next     ⌕ Forward  ⌕ Next Word ⌕ End
```

```

    args[i] = NULL;

    /* Create Child Process */
    pid_t pid = fork();

    if (pid < 0) {
        write(STDOUT_FILENO, "Fork failed\n", 12);
    }

    else if (pid == 0) {
        /* Child executes command */
        execvp(args[0], args);

        perror("Command failed");
        exit(1);
    }

    else {
        /* Parent waits */
        wait(NULL);
    }
}

/* Restore terminal settings */
tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}

```

```

harshitha18@LAPTOP-6E22AR6C:~$ nano task4.c
harshitha18@LAPTOP-6E22AR6C:~$ gcc task4.c -o task4
harshitha18@LAPTOP-6E22AR6C:~$ ./task4
Command > pwd
/home/harshitha18
Command > ls
OSSP      exec_example.c  file2.txt      hello.c        task4.c
backup.txt file1.txt        fork_example   process.c
exec_example file1.txt        fork_example.c task4
Command > echo Hello
Hello
Command > pwd
/home/harshitha18
Command > exit
harshitha18@LAPTOP-6E22AR6C:~$ |

```

The program successfully stores command history and recalls previous or next commands using the Up and Down arrow escape sequences. It also updates the input buffer and executes recalled commands using `fork()` and `execvp()`, providing basic shell-like history functionality.

Task-5;

```
harshitha18@LAPTOP-6E22AR6C:~$ nano task5.c
```

```
harshitha18@LAPTOP-6E22AR X + task5.c *
GNU nano 8.7.1
#define __GNU_SOURCE

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/mman.h>

#define INITIAL_SIZE 100
#define NEW_SIZE 200

struct Node {
    int data;
    struct Node *next;
};

int main() {
    char *buffer;

    /* Allocate 100 bytes using mmap() */
    buffer = mmap(NULL, INITIAL_SIZE,
        PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_ANONYMOUS,
        -1, 0);

    if (buffer == MAP_FAILED) {
        perror("mmap failed");
        return 1;
    }

    write(STDOUT_FILENO, "Enter text: ", 12);

    /* Read input safely */
    ssize_t bytesRead = read(STDIN_FILENO,
        buffer,
        INITIAL_SIZE - 1);

    if (bytesRead < 0) {
        perror("read failed");
        munmap(buffer, INITIAL_SIZE);
        return 1;
    }

    /* Remove newline */
    if (bytesRead > 0 && buffer[bytesRead - 1] == '\n') {
        buffer[bytesRead - 1] = '\0';
    }

    Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket Previous Back
    Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was Next Forward
```

```
harshitha18@LAPTOP-6E22AR X + task5.c *
GNU nano 8.7.1
    bytesRead--;
}
else {
    buffer[bytesRead] = '\0';
}

/* Display original input */
write(STDOUT_FILENO, "Original Input: ", 16);
write(STDOUT_FILENO, buffer, strlen(buffer));
write(STDOUT_FILENO, "\n", 1);

/* Resize memory from 100 bytes to 200 bytes */
char *newBuffer = mremap(buffer,
    INITIAL_SIZE,
    NEW_SIZE,
    MREMAP_MAYMOVE);

if (newBuffer == MAP_FAILED) {
    perror("mremap failed");
    munmap(buffer, INITIAL_SIZE);
    return 1;
}

buffer = newBuffer;

/* Append text safely */
const char *message = " - Memory Resized";
size_t currentLength = strlen(buffer);
size_t messageLength = strlen(message);

if (currentLength + messageLength < NEW_SIZE) {
    strcat(buffer, message);
}

/* Display updated string */
write(STDOUT_FILENO, "Updated Input: ", 15);
write(STDOUT_FILENO, buffer, strlen(buffer));
write(STDOUT_FILENO, "\n\n", 2);

/* Create Linked List with 5 nodes */
struct Node *head = NULL;
struct Node *temp = NULL;

for (int i = 1; i <= 5; i++) {
    struct Node *newNode =
```



```
harshitha18@LAPTOP-6E22A8  X  +  v  task5.c *
GNU nano 8.7.1
(struct Node *)malloc(sizeof(struct Node));

if (newNode == NULL) {
    perror("malloc failed");
    break;
}

newNode->data = i * 10;
newNode->next = NULL;

if (head == NULL) {
    head = newNode;
} else {
    temp->next = newNode;
}

temp = newNode;
}

/* Display Linked List */
printf("Linked List: ");

temp = head;

while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}

printf("\n");

/* Free Linked List */
temp = head;

while (temp != NULL) {
    struct Node *nextNode = temp->next;

    free(temp);

    temp = nextNode;
}

head = NULL;

/* Release mmap memory */
|
```

```
/* Release mmap memory */
if (munmap(buffer, NEW_SIZE) == -1) {
    perror("munmap failed");
    return 1;
}

printf("Memory released successfully.\n");

return 0;
}
|
```

```

harshitha18@LAPTOP-6E22AR6C:~$ gcc task5.c -o task5
harshitha18@LAPTOP-6E22AR6C:~$ ./task5
Enter text: Hello OSSP
Original Input: Hello OSSP
Updated Input: Hello OSSP - Memory Resized

Linked List: 10 20 30 40 50
Memory released successfully.
harshitha18@LAPTOP-6E22AR6C:~$ valgrind --leak-check=full ./task5
==1242== Memcheck, a memory error detector
==1242== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1242== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1242== Command: ./task5
==1242==
Enter text: Hello OSSP
Original Input: Hello OSSP
Updated Input: Hello OSSP - Memory Resized

Linked List: 10 20 30 40 50
Memory released successfully.
==1242==
==1242== HEAP SUMMARY:
==1242==    in use at exit: 0 bytes in 0 blocks
==1242==   total heap usage: 6 allocs, 6 frees, 1,104 bytes allocated
==1242==
==1242== All heap blocks were freed -- no leaks are possible
==1242==
==1242== For lists of detected and suppressed errors, rerun with: -s
==1242== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
harshitha18@LAPTOP-6E22AR6C:~$ |

```

The program successfully demonstrates dynamic memory allocation, buffer resizing, safe input handling, and linked-list memory management.

All dynamically allocated memory is properly released using `free()` and `munmap()`, and memory leaks can be verified using Valgrind.